

TokenTrail

Track, price and compare what your coding agents really cost

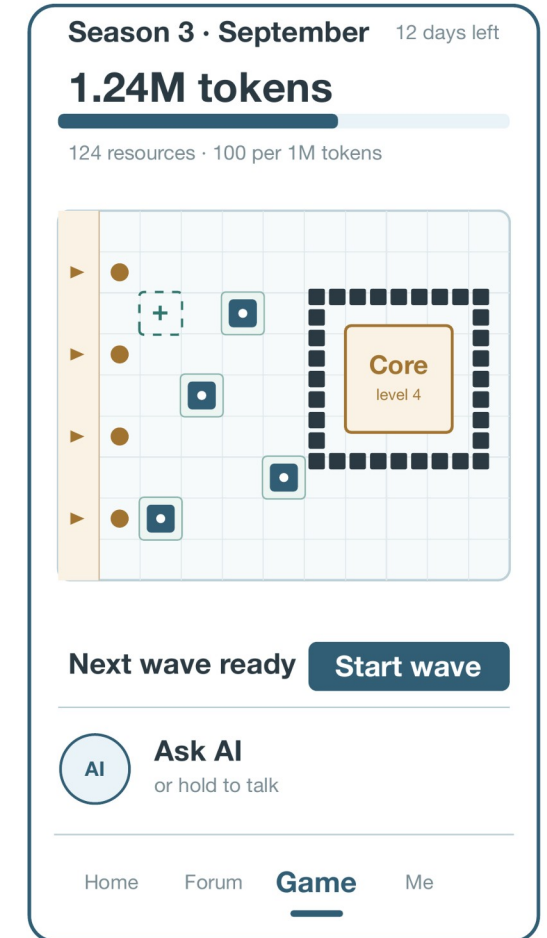
Category: Developer Productivity / API Cost Analytics

Target users: Students and independent developers who use API-key coding agents

Stack: Android · Java/XML · Room · Java advice service on the team's own server

Group 20 · Zhang Li (24107757) · Wang Tingdong (24107759) · Liu Zongrun (24107745)

Repository: github.com/LiuZongrun7/Mobile_Group_20



The six points the outline slides must cover

Guideline point	What the slides cover	Where
1. Brief app idea description	What TokenTrail is, who it is for, and what is out of scope	Slide 3
2. Discussion of similar / related apps	Codex, ZCode, DSH with dsh-context, LiteLLM and dsh-pet	Slide 5
3. Related apps: features and components	What each one covers, and the gap it leaves open	Slide 6
4. Open-source code available to reuse	Libraries, samples and licences we will record	Slide 7
5. Proposed features and uniqueness	Analytics and the home-screen assistant as the core; forum and game as supporting tabs	Slides 8-13
6. Approach and work plan to week 15	Alpha, Beta and Final checkpoints with acceptance tests	Slides 14-15

The written outline covers the same six points in the same order; this deck is the visual version of it.

An API-cost analytics app that makes agent spend visible

TokenTrail is an Android API-cost app for people who use several coding agents: Codex with OpenAI, ZCode with Xiaomi MiMo and DSH with DeepSeek. Its analytics home screen prices the pay-as-you-go calls those agents already logged, and an assistant on the same screen answers cost questions with evidence.

API-cost analytics (home)

Prices logged pay-as-you-go calls from Codex, ZCode and DSH at dated official rates, with source, coverage and session detail wherever the logs support it.

Home-screen AI assistant

An "Ask AI / hold to talk" entry above the navigation. Type, or dictate and confirm the transcript; replies cite evidence and admit missing data. Its own API cost is kept separate.

Supporting tabs

Forum: official model news plus user posts and tips. Game: a monthly tower-defence season built from tokens already logged. Both are extensions, not the main experience.

Scope rule. API-cost analytics and the home-screen assistant are core; forum and game are supporting extensions. Token Plan and other subscriptions are excluded, and TokenTrail never runs, routes or controls a coding agent.

Estimated, never invoiced

An API key alone cannot recover an agent's history, so a call is priced only when a saved rate snapshot covers its date. Missing logs or rates stay visible, and every monetary value is an estimate.

Why this is worth building

Each console sees one account

Codex, ZCode and DSH bill separately, in their own units, with no cross-provider total and no common price version. A user running three agents has three bills and no single monthly figure.

Tokens are not money

A response reports tokens for one call, not the agent's whole history. A call can only be priced when a saved rate snapshot covers its date, so some calls stay Unavailable rather than being guessed at.

Rating your own work is bad input

Asking users to score a task costs them extra effort and produces opinions, not measurements. It also makes the numbers depend on mood instead of on the data.

Spend leaves no trace

A day of agent work leaves nothing visible behind, so usage never turns into progress the user can see and nothing brings them back to the tools.

What TokenTrail does instead

It computes only what the data supports — logged token counts, dated official rates, cache behaviour and duration — and labels every figure Estimated, Partial estimate or Unavailable. It never asks the user to judge how good an answer was.

Five products we studied before writing a line of code

FIRST-PARTY AGENT

Codex

OpenAI's coding agent. Can run with an OpenAI API key and be billed through the Platform account.

The OpenAI-side agent we track.

MODEL PROVIDER

ZCode

Supports Xiaomi MiMo as a model provider. Usable MiMo API logs are imported; Token Plan usage is excluded.

Tracked through its MiMo API endpoint.

SESSION PLUGIN

DSH / dsh-context

DSH runs coding sessions; its dsh-context plugin visualises tokens, cache and activity.

Import its logs into Android; no live agent control.

GATEWAY

LiteLLM

Open-source gateway with spend tracking and budgets — but only for traffic that goes through it.

Teaches the vocabulary of spend tracking and budgets.

COMPANION

dsh-pet

Floating companion driven by DSH session events, with touch and feeding actions. Closest thing to a game.

Its entry point inspires our assistant panel on the home screen.

What the review told us

Every product above is strong inside its own boundary: a first-party console prices one provider, a plugin reads one harness, a gateway sees only proxied traffic. None of them prices three providers from records the user already owns, and only dsh-pet treats usage as something to come back to.

The review also set our boundaries: no proxying, no subscription quotas, and no overlay on another app.

The opportunity

The outline names three contributions:

- unified analytics over logs the user owns;
- reproducible costs that keep the price version;
- evidence-based advice that admits missing data.

Section 2 of the written outline compares the same five products, with a link to each one.

Component by component: what they cover, what is left open

Product	Relevant capability	Gap addressed by TokenTrail
Codex	API-key coding agent billed through OpenAI Platform.	Dated estimates from the available Codex logs.
ZCode	Supports Xiaomi MiMo as a model provider.	Import usable MiMo API logs; exclude Token Plan usage.
DSH / dsh-context	DSH runs coding sessions; its context plugin visualises tokens, cache and activity.	Import usable DSH logs into Android; no live agent control.
LiteLLM	Gateway spend tracking and budgets for proxied calls.	Imports records without proxying agent traffic.
dsh-pet	Floating companion driven by DSH session events.	Interactive home-screen assistant with questions and evidence-linked advice.

Names link to each product's own documentation or repository. The full comparison is section 2 of the written outline.

What we reuse, and what we build ourselves

Layer	Planned tools and rationale
Android interface	Java/XML, Material Components, Navigation and ViewModel/LiveData; RecyclerView lists, speech-to-text and typed fallback.
Network and work	Retrofit/OkHttp connects the Java advice service; bounded read-only tools and one model API. WorkManager checks prices.
Persistence and security	Room stores usage, prices and history; the team's own server supports sync and public posts. The service authenticates every request and scopes every read to the caller's account; no client holds a model key.
Open-source reuse	Room (Apache-2.0) for local records; MPAndroidChart (Apache-2.0) for charts, added as a dependency; Room with a View is an archived architecture reference with no copied code. dsh-context/LiteLLM: comparison only; dsh-pet: entry inspiration only. The game engine is original.

Original or properly licensed art only. Every reused file and its licence, version and source is recorded in the repository, and this layer is revisited if a dependency changes the plan.

Section 8 of the written outline lists the same four layers, with a licence link for each reused library.

Three contributions, and the rule that keeps them honest

Unified analytics

De-duplicates authorised logs and shows source, coverage and session detail — but only where the records support it.

Where a session view is not supported, it shows a provider-level estimate instead of inventing one.

Reproducible costs

Java applies dated official rates by provider, model, token class and service tier, and keeps the price version behind every estimate.

New rates apply prospectively; old estimates keep the version they were computed with.

Evidence-based AI advice

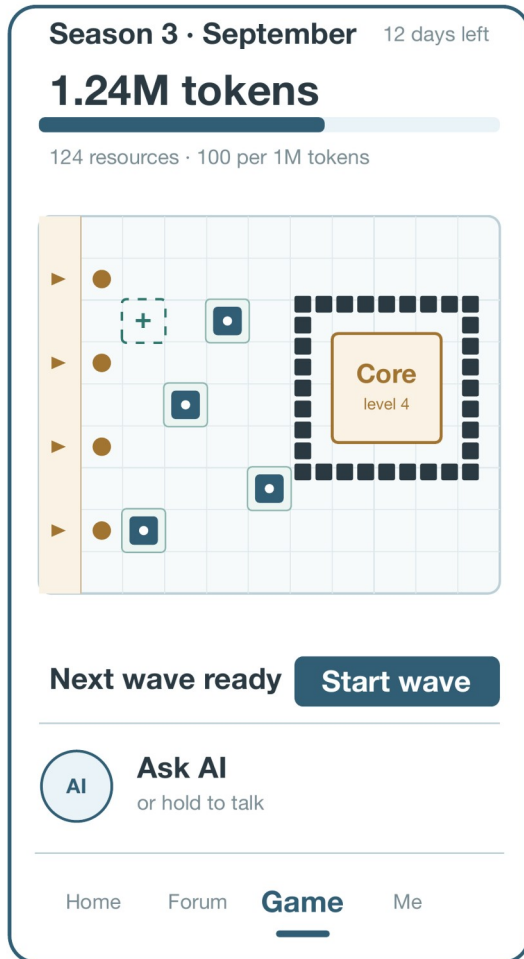
Text or speech questions obtain calculated evidence from read-only tools; replies disclose sample size and missing data.

It compares recorded cost, tokens and cache use — never answer quality, and it asks for no task ratings.

Scope rule: API-cost analytics and the home-screen assistant are core; forum and game are supporting extensions. Live cross-agent status and system-wide overlays are outside the first release.

Values are labelled. Every monetary figure is Estimated, Partial estimate or Unavailable — a cost that cannot be computed is never shown as zero. Source-billed amounts are kept as separate reconciliation evidence, not as replacements for the estimates.

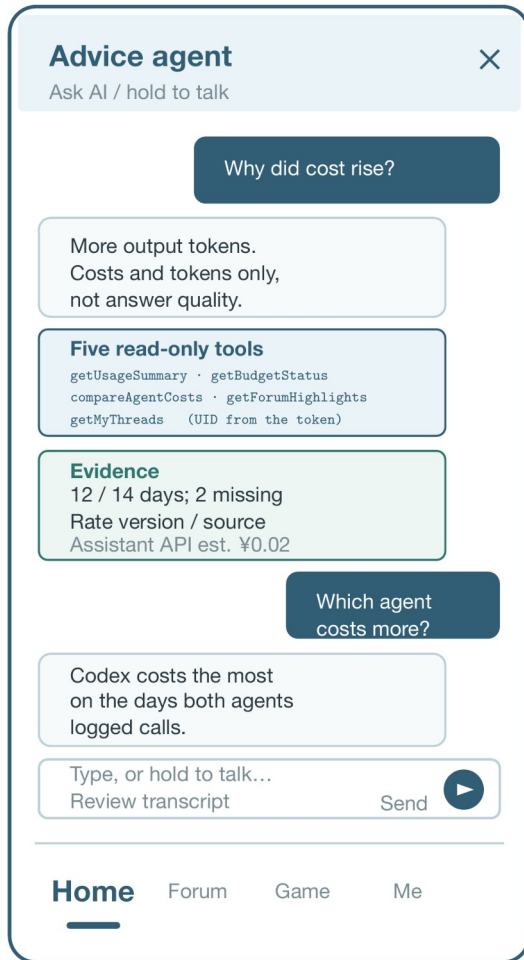
A monthly tower-defence season built from tokens already logged



- **It is a supporting tab.** Reached from the bottom bar, beside the forum. It is played in monthly seasons that match the billing cycle the provider subscriptions already follow.
- **Tokens become resources.** Input, cache and output tokens already logged convert into separate resource pools at a fixed rate per million tokens.
- **You plan the space yourself.** Towers and wall segments are placed and upgraded by hand on a build grid around the data-centre core, which expands in one direction.
- **Enemies arrive from one side.** A wave enters from a single edge, so where each tower stands and whether the walls close decide how the wave goes.
- **A wave starts when you start it.** Waves run in real time, so no offline simulation and no scheduled settlement is needed. The season resets each month.

Why it matters. A day of agent work leaves a visible result without asking the user to spend differently. Playing never changes usage, cost or budget totals, and re-importing the same logs grants no duplicate resources — so the balance cannot be reached on invented data.

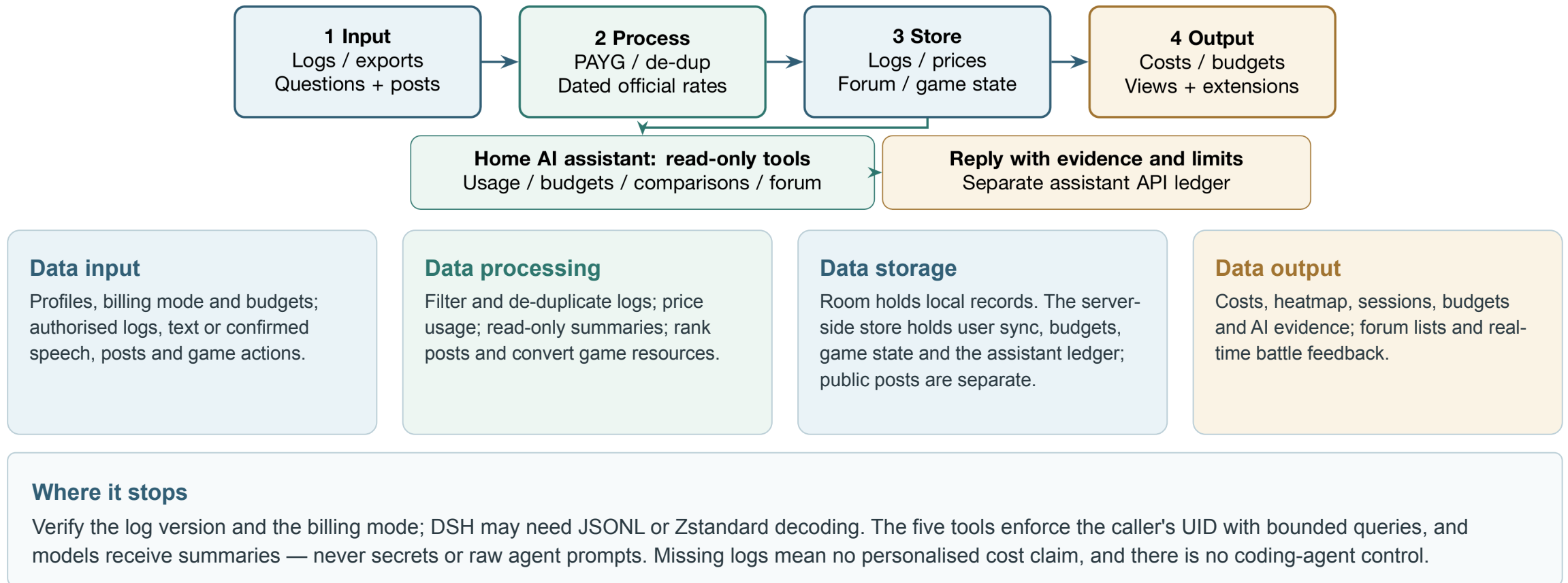
An assistant that answers from evidence, not opinion



- **It lives on the home screen.** An "Ask AI / hold to talk" entry above the navigation opens an expandable panel on the analytics dashboard, where the figures it reasons about already are. It reads your logged usage and recent forum posts.
- **Ask by text, or hold to talk.** Dictate a question, then review or edit the transcript before sending. The microphone is requested when used; denial or failure leaves typing available, and raw audio is not retained by default.
- **A Java service does the work.** It calls one selected OpenAI, Xiaomi MiMo or DeepSeek pay-as-you-go model, which may request tools through function calling. Java executes and validates every call.
- **Five read-only tools.** getUsageSummary · getBudgetStatus · compareAgentCosts · getForumHighlights · getMyThreads. The UID comes from the signed session token, never from a parameter.
- **It admits what it does not know.** Replies state evidence, price versions and missing data; no logs means no cost-based answer. Its own tokens and cost sit in a separate ledger, excluded from coding-agent totals and game resources.

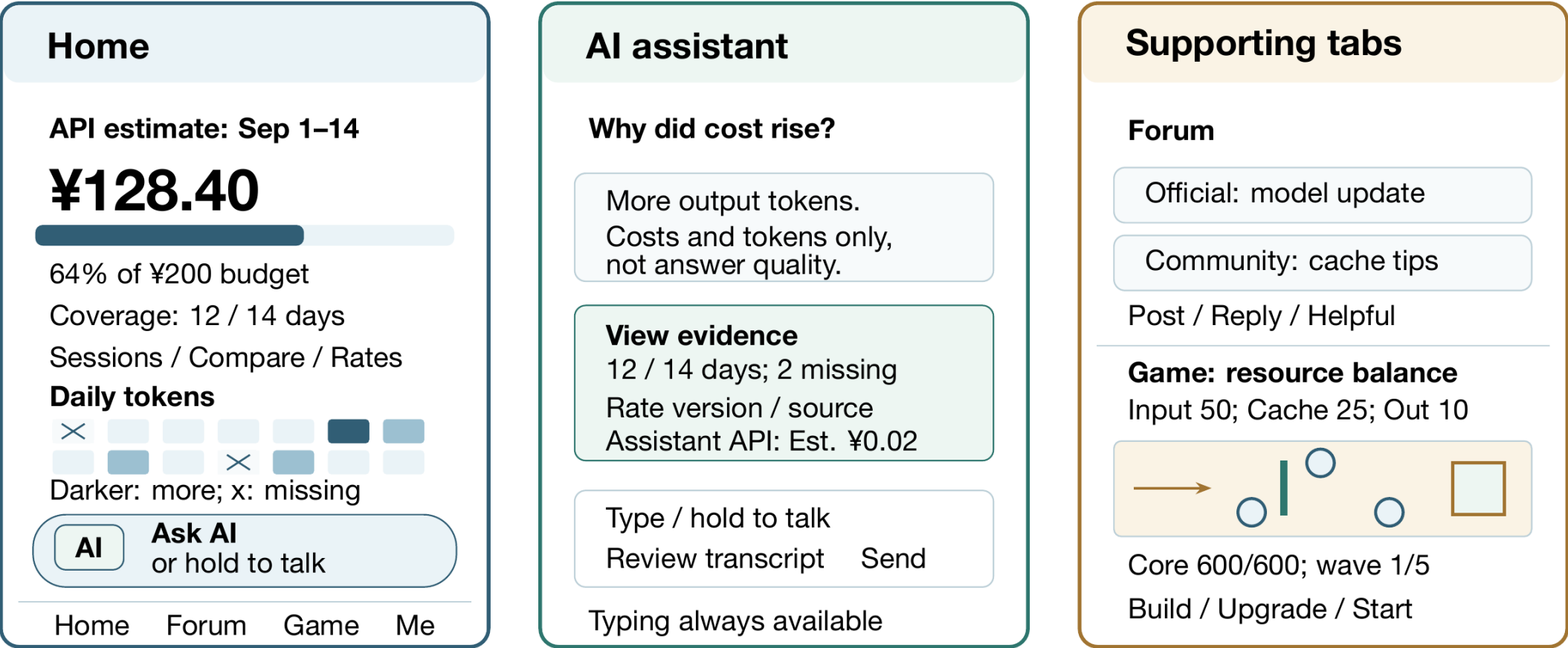
Boundary. It can recommend but cannot run, route or interrupt a coding agent, and cannot enforce a hard spending cap. It compares cost and token use, never answer quality.

One path from logs to the app, and the four data aspects



The same four aspects (input, processing, storage, output) run through the written outline, the implementation report, the code comments and the final demonstration.

Home leads with analytics and the assistant entry



Tap "Ask AI" to open the panel, or hold to dictate and confirm the transcript. Sessions expose source, date and token classes only where logs support them, and comparisons use matching filters. Forum and Game are supporting tabs, and assistant API costs are excluded from the home total.

Functional requirements mapped to what already exists

Ser	Existing feature	TokenTrail improvement / addition	Type
1	Cost dashboards	Cross-provider dated estimates with source and coverage	Improved
2	Session viewers	Log-supported sessions and token-class breakdowns	Improved
3	Billing budgets	Cross-provider estimated budget and advisory alerts	Improved
4	AI assistants	Home AI: text and voice, usage evidence and a separate API ledger	New
5	Forums	Linked official news, posts and replies, and context for the advice	Improved
6	Tower defence	Monthly game resources from verified coding-agent API logs	New

Six rows, in the same order as section 11 of the written outline. Only rows 4 and 6 are new; the rest improve a view the providers already offer one account at a time.

Checkpoints from this week to the final submission

Course weeks	Work	Checkpoint / acceptance
1-3	Verify Codex/ZCode/DSH samples, billing modes, rate dimensions and UI; define access rules.	Verified sources; isolated accounts.
4-7	Build import, storage, dated pricing and analytics; define the five read-only tool contracts.	Correct rates; idempotent import.
8-9 · Alpha	Login → profile → import → cost → home.	Separate real and sample data; source, GitHub link and MP4.
10-12 · Beta	Wang: assistant and forum using Zhang's queries. Liu: settlement and the real-time game.	Cited evidence; typing fallback; ranked posts; smooth waves.
13-15 · Final	Validate advice, cost separation, game balance, accessibility and isolation; complete the documentation.	Demo, tests, report and video; licence records.

The team commits to GitHub throughout and adjusts scope based on tested progress.

Who builds what, and how we will know it works

Zhang Li · 24107757

Authentication, log import and storage, dated pricing, budgets, the dashboard and the data-query interfaces.

Wang Tingdong · 24107759

The home-screen AI assistant, its text/voice panel, the Java advice service, the five read-only tools and the assistant API ledger; forum news, posts, replies and ranking.

Liu Zongrun · 24107745

The tower-defence engine and UI, resource settlement, monthly state, placement and upgrades, and enemy waves.

Demo path

Account isolation → tracked Codex, ZCode and DSH profiles → dated estimates from token logs → heatmap and budget warning → "Why did cost rise?" asked by text or confirmed speech, answered with evidence and its own assistant API cost → forum posts and replies → a player-started game wave.

Validation

Test plan exclusion, log formats, token classes, dated and time-based rates, missing data, de-duplication and account/tool isolation. Replies are checked against evidence, text fallback and ledger separation, and re-importing logs grants no duplicate game resources. Sample data never enters real totals.

Repository: https://github.com/LiuZongrun7/Mobile_Group_20 · UI/UX, accessibility and integration are shared across all three members.